Assignment-22.3

Task 1 – Bias Detection in AI Suggestions (Alternative Example)

Example Scenario:

Ask AI to generate a resume shortlisting algorithm multiple times.

• Change prompt wording (e.g., “select best candidate” vs “ensure diversity”).

• Compare outputs and logic

```python
import json
candidates = [
    {"name": "Ravi", "college_tier": 1, "experience": 5, "skills_score": 5, "gender": "male", "background": "indian", "potential": 4},
    {"name": "Priya", "college_tier": 3, "experience": 4, "skills_score": 5, "gender": "female", "background": "indian", "potential": 5},
    {"name": "Ahmed", "college_tier": 2, "experience": 0, "skills_score": 5, "gender": "male", "background": "other", "potential": 5},
    {"name": "Sneha", "college_tier": 3, "experience": 2, "skills_score": 4, "gender": "female", "background": "indian", "potential": 5},
    {"name": "John", "college_tier": 4, "experience": 6, "skills_score": 4, "gender": "male", "background": "foreign", "potential": 3},
    {"name": "Meena", "college_tier": 2, "experience": 3, "skills_score": 4, "gender": "female", "background": "indian", "potential": 4},
]
# Tier: 1 best, 4 foreign
def neutral_score(c):
    edu = 5 - c["college_tier"]
    exp = c["experience"]
    skill = c["skills_score"]
    return edu + exp + skill
def best_candidate_score(c):
    edu = (5 - c["college_tier"]) * 3  # prioritize
    exp = c["experience"] * 2
    skill = c["skills_score"] * 2
    return edu + exp + skill
def diversity_score(c):
    # avoid college bias, consider skills and potential equally
    return c["skills_score"] + c["potential"]
def inclusive_score(c):
    # balance exp, skills, diversity
    # diversity: bonus for underrepresented
    base = c["experience"] + c["skills_score"]
    diversity_bonus = 0
    if c["gender"] == "female": diversity_bonus += 1
    if c["background"] != "indian": diversity_bonus += 1
    return base + diversity_bonus
def rank_candidates(score_func):
    scored = [(c["name"], score_func(c)) for c in candidates]
    scored.sort(key=lambda x: x[1], reverse=True)
    return [name for name, score in scored]
print("Neutral Approach Rankings:")
neutral_rank = rank_candidates(neutral_score)
for i, name in enumerate(neutral_rank, 1):
    print(f"{i}. {name}")
print("Logic: Scores based on education tier (higher tier better), experience years, and skills score. Total score = edu + exp + skill.")
print("\nBest Candidate Approach Rankings:")
best_rank = rank_candidates(best_candidate_score)
for i, name in enumerate(best_rank, 1):
    print(f"{i}. {name}")
print("Logic: Prioritizes top universities (triple weight on education), high experience (double), strong skills (double). Total score = 3*edu + 2*exp + 2*skill.")
print("\nDiversity-Focused Approach Rankings:")
diversity_rank = rank_candidates(diversity_score)
for i, name in enumerate(diversity_rank, 1):
    print(f"{i}. {name}")
print("Logic: Ignores college tier, gender, background. Considers skills and potential equally. Total score = skill + potential.")
print("\nInclusive Hiring Approach Rankings:")
inclusive_rank = rank_candidates(inclusive_score)
for i, name in enumerate(inclusive_rank, 1):
    print(f"{i}. {name}")
print("Logic: Balances experience and skills, adds diversity bonus for females and non-Indian backgrounds. Total score = exp + skill + diversity_bonus.")
```

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING> & "c:/Users/gbsgu/OneDrive/Documents/Desktop/Ai aSSISTED CODEING/.venv/Scripts/python.exe" "c:/Users/gbsgu/OneDrive/Documents/Desk
Neutral Approach Rankings:
1. Ravi
2. Priya
3. John
4. Meena
5. Ahmed
6. Sneha
Logic: Scores based on education tier (higher tier better), experience years, and skills score. Total score = edu + exp + skill.

Best Candidate Approach Rankings:
1. Ravi
2. Priya
3. John
4. Meena
5. Ahmed
6. Sneha
Logic: Prioritizes top universities (triple weight on education), high experience (double), strong skills (double). Total score = 3*edu + 2*exp + 2*skill.

Diversity-Focused Approach Rankings:
1. Priya
2. Ahmed
3. Ravi
4. Sneha
5. Meena
6. John
Logic: Ignores college tier, gender, background. Considers skills and potential equally. Total score = skill + potential.

Inclusive Hiring Approach Rankings:
1. John
2. Ravi
3. Priya
4. Meena
5. Sneha
6. Ahmed
Logic: Balances experience and skills, adds diversity bonus for females and non-Indian backgrounds. Total score = exp + skill + diversity_bonus.
```

Task 2 – Privacy and Sensitive Data Handling (Alternative Example)

Example Scenario:

Ask AI to write code for processing student academic records (roll number, marks, email).

Write a secure Python program to process student academic records (roll number, marks, email) while ensuring privacy by masking sensitive data, applying encryption where needed, and following data protection principles like FERPA and GDPR. Also explain the ethical responsibilities of developers when using AI-generated code.

```
Safe Display:
{'roll_number': '1***5', 'marks': 'Encrypted', 'email': 's*******@u*********.edu'}
Decrypted (authorized access only):
Roll: 12345, Marks: 85.5, Email: student1@university.edu
---
Safe Display:
{'roll_number': '6***0', 'marks': 'Encrypted', 'email': 's*******@u*********.edu'}
Decrypted (authorized access only):
Roll: 67890, Marks: 92.0, Email: student2@university.edu
---
Record deleted.
```

Task 3 – AI-Generated Security Risks (Alternative Example)

Example Scenario:

Use AI to generate a payment processing script for an online shopping system.

Generate a secure payment processing script for an online shopping system and analyze potential security risks such as plain-text card data storage and missing input validation. Suggest improvements using secure coding practices, encryption, and compliance with standards like PCI DSS. Also explain ethical coding responsibilities to prevent financial fraud and misuse.

```python
import requests
from cryptography.fernet import Fernet
import os
import re

# Mock encryption key (in production, use secure key management)
KEY = Fernet.generate_key()
cipher = Fernet(KEY)

class SecurePaymentProcessor:
    def __init__(self, api_key: str):
        self.api_key = api_key  # Store securely, e.g., env var
        self.base_url = "https://api.stripe.com/v1"  # Mock Stripe API

    def validate_card_number(self, card_number: str) -> bool:
        # Luhn algorithm for validation
        def luhn_checksum(card_num: str) -> bool:
            def digits_of(n):
                return [int(d) for d in str(n)]
            digits = digits_of(card_num)
            odd_digits = digits[-1::-2]
            even_digits = digits[-2::-2]
            checksum = sum(odd_digits)
            for d in even_digits:
                checksum += sum(digits_of(d * 2))
            return checksum % 10 == 0
        # Remove spaces and dashes
        card_number = re.sub(r'[\s-]', '', card_number)
        return len(card_number) in [13, 15, 16, 19] and card_number.isdigit() and luhn_checksum(card_number)

    def validate_expiry(self, expiry: str) -> bool:
        # MM/YY format
        if not re.match(r'^(0[1-9]|1[0-2])\/\d{2}$', expiry):
            return False
        month, year = expiry.split('/')
        return 1 <= int(month) <= 12 and int(year) >= 24  # Assume current year 24+

    def validate_cvc(self, cvc: str) -> bool:
        return re.match(r'^\d{3,4}$', cvc) is not None

    def tokenize_card(self, card_number: str, expiry: str, cvc: str) -> str:
        # Simulate tokenization via API (do not send real card data)
        payload = {
            'card[number]': card_number,
            'card[exp_month]': expiry.split('/')[0],
            'card[exp_year]': '20' + expiry.split('/')[1],
            'card[cvc]': cvc
        }
        headers = {'Authorization': f'Bearer {self.api_key}'}
        response = requests.post(f"{self.base_url}/tokens", data=payload, headers=headers)
        if response.status_code == 200:
            return response.json()['id']  # Token
        else:
            raise ValueError("Tokenization failed")

    def encrypt_token(self, token: str) -> str:
        return cipher.encrypt(token.encode()).decode()

    def decrypt_token(self, encrypted_token: str) -> str:
        return cipher.decrypt(encrypted_token.encode()).decode()

    def process_payment(self, amount: float, currency: str, token: str) -> dict:
        # Charge using token
        payload = {
            'amount': int(amount * 100),  # Cents
            'currency': currency,
            'source': token
```

```python
class SecurePaymentProcessor:
    def tokenize_card(self, card_number: str, expiry: str, cvc: str) -> str:
        payload = {
            'card[number]': card_number,
            'card[exp_month]': expiry.split('/')[0],
            'card[exp_year]': '20' + expiry.split('/')[1],
            'card[cvc]': cvc
        }
        headers = {'Authorization': f'Bearer {self.api_key}'}
        response = requests.post(f"{self.base_url}/tokens", data=payload, headers=headers)
        if response.status_code == 200:
            return response.json()['id']  # Token
        else:
            raise ValueError("Tokenization failed")

    def encrypt_token(self, token: str) -> str:
        return cipher.encrypt(token.encode()).decode()

    def decrypt_token(self, encrypted_token: str) -> str:
        return cipher.decrypt(encrypted_token.encode()).decode()

    def process_payment(self, amount: float, currency: str, token: str) -> dict:
        # Charge using token
        payload = {
            'amount': int(amount * 100),  # Cents
            'currency': currency,
            'source': token
        }
        headers = {'Authorization': f'Bearer {self.api_key}'}
        response = requests.post(f"{self.base_url}/charges", data=payload, headers=headers)
        return response.json()

# Example usage
def main():
    processor = SecurePaymentProcessor("sk_test_mock_key")  # Use real key securely

    # Simulate user input (in production, from secure form)
    card_number = "4111111111111111"  # Test card
    expiry = "12/25"
    cvc = "123"

    if not (processor.validate_card_number(card_number) and
            processor.validate_expiry(expiry) and
            processor.validate_cvc(cvc)):
        print("Invalid card details")
        return

    try:
        token = processor.tokenize_card(card_number, expiry, cvc)
        encrypted_token = processor.encrypt_token(token)
        print(f"Encrypted Token: {encrypted_token}")

        # Store encrypted token securely (e.g., database)
        # Later, decrypt for payment
        decrypted_token = processor.decrypt_token(encrypted_token)
        result = processor.process_payment(100.0, "usd", decrypted_token)
        print("Payment Result:", result)
    except ValueError as e:
        print(e)

if __name__ == "__main__":
    main()
```

```
PS C:\Users\gbsgu\OneDrive\Do
Tokenization failed
PS C:\Users\gbsgu\OneDrive\Do
```

Task 4 – Accountability in Human–AI Collaboration (Alternative Example)

Scenario:

AI generates code for an autonomous traffic signal control system.

Discussion Questions:

- If a software error causes accidents, who is accountable?

Generate a Python program simulating an autonomous traffic signal control system. Include logic for signal timing and decision-making. Add error handling to prevent unsafe states (e.g., two directions getting green at the same time), include logging for accountability, and comments explaining how developers, AI systems, and authorities share responsibility in case of failures.

```python
import logging
import threading
import time
from enum import Enum, auto

# Configure logging for traceability and accountability.
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)


class SignalState(Enum):
    RED = auto()
    YELLOW = auto()
    GREEN = auto()


class Direction(Enum):
    NORTH_SOUTH = "North-South"
    EAST_WEST = "East-West"


class UnsafeStateError(Exception):
    """Raised when two directions would be green at the same time."""
    pass


class TrafficSignalController:
    """A simple autonomous traffic signal controller simulation."""

    def __init__(self, cycle_time=30, yellow_time=5):
        self.cycle_time = cycle_time
        self.yellow_time = yellow_time
        self.current_direction = Direction.NORTH_SOUTH
        self.states = {
            Direction.NORTH_SOUTH: SignalState.GREEN,
            Direction.EAST_WEST: SignalState.RED,
        }
        self.lock = threading.Lock()
        self.running = False
        logging.info("TrafficSignalController initialized")

    def validate_states(self):
        """Ensure that only one direction has GREEN at a time."""
        green_directions = [d for d, s in self.states.items() if s == SignalState.GREEN]
        if len(green_directions) > 1:
            raise UnsafeStateError(
                f"Unsafe state detected: multiple green signals {green_directions}"
            )

    def set_state(self, direction, state):
        """Set the state for one direction and validate overall safety."""
        with self.lock:
```

```python
class TrafficSignalController:
    def __init__(self, cycle_time=30, yellow_time=5):
        self.states = {
            Direction.NORTH_SOUTH: SignalState.GREEN,
            Direction.EAST_WEST: SignalState.RED,
        }
        self.lock = threading.Lock()
        self.running = False
        logging.info("TrafficSignalController initialized")

    def validate_states(self):
        """Ensure that only one direction has GREEN at a time."""
        green_directions = [d for d, s in self.states.items() if s == SignalState.GREEN]
        if len(green_directions) > 1:
            raise UnsafeStateError(
                f"Unsafe state detected: multiple green signals {green_directions}"
            )

    def set_state(self, direction, state):
        """Set the state for one direction and validate overall safety."""
        with self.lock:
            logging.debug("Setting %s to %s", direction.value, state.name)
            self.states[direction] = state
            self.validate_states()
            logging.info(
                "Updated state: %s=%s, %s=%s",
                Direction.NORTH_SOUTH.value,
                self.states[Direction.NORTH_SOUTH].name,
                Direction.EAST_WEST.value,
                self.states[Direction.EAST_WEST].name,
            )

    def switch_directions(self):
        """Switch control from one direction to the other."""
        with self.lock:
            if self.current_direction == Direction.NORTH_SOUTH:
                self.set_state(Direction.NORTH_SOUTH, SignalState.YELLOW)
                time.sleep(self.yellow_time)
                self.set_state(Direction.NORTH_SOUTH, SignalState.RED)
                self.set_state(Direction.EAST_WEST, SignalState.GREEN)
                self.current_direction = Direction.EAST_WEST
            else:
                self.set_state(Direction.EAST_WEST, SignalState.YELLOW)
                time.sleep(self.yellow_time)
                self.set_state(Direction.EAST_WEST, SignalState.RED)
                self.set_state(Direction.NORTH_SOUTH, SignalState.GREEN)
                self.current_direction = Direction.NORTH_SOUTH

    def run_cycle(self):
        """Run one full timing cycle for the current green direction."""
        green_direction = self.current_direction
        logging.info("Signal cycle begins for %s", green_direction.value)
        time.sleep(self.cycle_time - self.yellow_time)
        self.switch_directions()
        logging.info("Signal cycle ended for %s", green_direction.value)

    def start(self, cycles=3):
        """Start the autonomous signal controller simulation."""
        self.running = True
        logging.info("Starting traffic signal controller for %d cycles", cycles)
        try:
            for cycle_number in range(1, cycles + 1):
                if not self.running:
                    break
                logging.info("Beginning cycle %d", cycle_number)
                self.run_cycle()
        except UnsafeStateError as ex:
```

```python
class TrafficSignalController:
    def switch_directions(self):
            time.sleep(self.yellow_time)
            self.set_state(Direction.EAST_WEST, SignalState.RED)
            self.set_state(Direction.NORTH_SOUTH, SignalState.GREEN)
            self.current_direction = Direction.NORTH_SOUTH

    def run_cycle(self):
        """Run one full timing cycle for the current green direction."""
        green_direction = self.current_direction
        logging.info("Signal cycle begins for %s", green_direction.value)
        time.sleep(self.cycle_time - self.yellow_time)
        self.switch_directions()
        logging.info("Signal cycle ended for %s", green_direction.value)

    def start(self, cycles=1):
        """Start the autonomous signal controller simulation."""
        self.running = True
        logging.info("Starting traffic signal controller for %d cycles", cycles)
        try:
            for cycle_number in range(1, cycles + 1):
                if not self.running:
                    break
                logging.info("Beginning cycle %d", cycle_number)
                self.run_cycle()
        except UnsafeStateError as ex:
            logging.error("Failure detected: %s", ex)
            # The system stops to prevent escalation and records the incident.
            self.running = False
        except Exception as ex:
            logging.exception("Unexpected error during signal operation")
            self.running = False
        finally:
            logging.info("Traffic signal controller stopped")

    def stop(self):
        """Stop the simulation gracefully."""
        self.running = False
        logging.info("Received stop request")


def main():
    """Main entrypoint for the traffic signal control simulation."""
    controller = TrafficSignalController(cycle_time=20, yellow_time=4)

    # Comments on shared responsibility in a safety-critical system:
    # - Developers are responsible for designing clear control logic, validating transitions, and implementing safe defaults.
    # - AI systems can support decision-making by monitoring traffic, predicting demand, and suggesting improved timings.
    # - Authorities are responsible for oversight, auditing logs, reviewing failures, and enforcing safety standards.
    # In the event of a fault, developers must fix control logic, AI systems should provide diagnostics,
    # and authorities must review incident records to assign accountability.

    try:
        controller.start(cycles=5)
    except KeyboardInterrupt:
        logging.warning("Simulation interrupted by user")
        controller.stop()


if __name__ == "__main__":
    main()
```

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING> & "c:\Users\gbsgu\OneDrive\Documents\Desk
(.venv) PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING> & "c:/Users/gbsgu/OneDrive/Docume
2026-04-08 10:29:58 [INFO] TrafficSignalController initialized
2026-04-08 10:29:58 [INFO] Starting traffic signal controller for 5 cycles
2026-04-08 10:29:58 [INFO] Beginning cycle 1
2026-04-08 10:29:58 [INFO] Signal cycle begins for North-South
```

Task 5 – Limitations of AI in Critical Systems (Alternative

Example)

Example Scenario:

Ask AI to generate code for an aircraft maintenance fault detection

System.

Generate a Python program that simulates an aircraft maintenance fault detection system. Include logic to detect faults from sensor data, apply validation checks, and raise alerts for critical issues. Add safeguards such as input validation, redundancy checks, and logging. Include comments highlighting the limitations of AI-generated code, the need for expert human validation, and how safety-critical systems should be reviewed and certified before deployment.

aircraft_maintenance_fault_detection.py > ...

```python
import logging
import random
from dataclasses import dataclass
from typing import Dict, List, Optional

# Configure logging for auditability and incident tracing.
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)


class ValidationError(Exception):
    """Raised when input sensor data does not meet validation requirements."""
    pass


class CriticalAlert(Exception):
    """Raised when a critical fault is detected and immediate action is required."""
    pass


@dataclass
class SensorReading:
    name: str
    value: float
    unit: str


class FaultDetectionSystem:
    """Simulates an aircraft maintenance fault detection system."""

    MIN_SENSOR_VALUE = -1000.0
    MAX_SENSOR_VALUE = 10000.0

    def __init__(self, redundancy_threshold: int = 2):
        self.redundancy_threshold = redundancy_threshold
        self.alert_history: List[str] = []
        logging.info("FaultDetectionSystem initialized with redundancy_threshold=%d", redundancy_threshold)

    def validate_input(self, reading: SensorReading) -> None:
        """Validate raw sensor input to ensure data is within expected bounds."""
        if reading.value is None:
            raise ValidationError(f"Sensor '{reading.name}' has missing value")
        if not isinstance(reading.value, (int, float)):
            raise ValidationError(f"Sensor '{reading.name}' value must be numeric")
        if not (self.MIN_SENSOR_VALUE <= reading.value <= self.MAX_SENSOR_VALUE):
            raise ValidationError(
                f"Sensor '{reading.name}' value {reading.value} {reading.unit} outside safe range"
            )
        logging.debug("Validated input for %s: %s %s", reading.name, reading.value, reading.unit)

    def detect_faults(self, readings: List[SensorReading]) -> Dict[str, str]:
        """Analyze sensor data and detect faults using rule-based checks."""
        self._ensure_redundancy(readings)
        fault_report: Dict[str, str] = {}

        for reading in readings:
            self.validate_input(reading)
            if reading.name == "engine_temperature" and reading.value > 950:
                fault_report[reading.name] = "Overheat"
            elif reading.name == "hydraulic_pressure" and reading.value < 1500:
                fault_report[reading.name] = "Low hydraulic pressure"
            elif reading.name == "vibration_level" and reading.value > 8:
                fault_report[reading.name] = "Excessive vibration"
            elif reading.name == "battery_voltage" and reading.value < 24:
                fault_report[reading.name] = "Low battery voltage"
            elif reading.name == "fuel_flow" and reading.value > 1200:
                fault_report[reading.name] = "Abnormal fuel flow"

        if fault_report:
            logging.warning("Detected faults: %s", fault_report)
        else:
            logging.info("No faults detected from current sensor data")

        return fault_report

    def _ensure_redundancy(self, readings: List[SensorReading]) -> None:
        """Check that redundant sensors are available for critical parameters."""
        sensor_counts: Dict[str, int] = {}
        for reading in readings:
            sensor_counts[reading.name] = sensor_counts.get(reading.name, 0) + 1

        for name, count in sensor_counts.items():
            if count < self.redundancy_threshold:
                raise ValidationError(
                    f"Insufficient redundancy for sensor '{name}': {count} reading(s) found"
                )
        logging.debug("Redundancy validation passed: %s", sensor_counts)

    def raise_alert(self, faults: Dict[str, str]) -> None:
        """Raise alerts for critical faults and log the incident."""
        for sensor, issue in faults.items():
            alert_message = f"CRITICAL ALERT: {sensor} - {issue}"
            self.alert_history.append(alert_message)
            logging.error(alert_message)

        if any(issue in ["Overheat", "Low hydraulic pressure", "Excessive vibration"] for issue in faults.values()):
            raise CriticalAlert("Critical aircraft maintenance fault detected")

    def analyze(self, readings: List[SensorReading]) -> Optional[str]:
        """Full evaluation of current sensor data with safeguards and alerts."""
        try:
            faults = self.detect_faults(readings)
            if faults:
                self.raise_alert(faults)
            return None
        except ValidationError as error:
            logging.error("Input validation failed: %s", error)
            return str(error)
        except CriticalAlert as alert:
            logging.exception("Critical fault escalation")
            return str(alert)
        except Exception as error:
            logging.exception("Unexpected error during fault detection")
            return str(error)


def simulate_sensor_data() -> List[SensorReading]:
    """Generate simulated sensor readings for a maintenance check."""
    # Note: simulated data is for demonstration only.
    return [
        SensorReading("engine_temperature", random.uniform(800, 900), "C"),
        SensorReading("engine_temperature", random.uniform(800, 900), "C"),
        SensorReading("hydraulic_pressure", random.uniform(1400, 1700), "psi"),
        SensorReading("hydraulic_pressure", random.uniform(1400, 1700), "psi"),
```

```python
    """Generate simulated sensor readings for a maintenance check."""
    # Note: Simulated data is for demonstration only.
    return [
        SensorReading("engine_temperature", random.uniform(800, 980), "C"),
        SensorReading("engine_temperature", random.uniform(800, 980), "C"),
        SensorReading("hydraulic_pressure", random.uniform(1400, 1700), "psi"),
        SensorReading("hydraulic_pressure", random.uniform(1400, 1700), "psi"),
        SensorReading("vibration_level", random.uniform(3, 9), "g"),
        SensorReading("vibration_level", random.uniform(3, 9), "g"),
        SensorReading("battery_voltage", random.uniform(22, 28), "V"),
        SensorReading("battery_voltage", random.uniform(22, 28), "V"),
        SensorReading("fuel_flow", random.uniform(900, 1250), "L/h"),
        SensorReading("fuel_flow", random.uniform(900, 1250), "L/h"),
    ]


def main():
    controller = FaultDetectionSystem(redundancy_threshold=2)

    # Safety-critical system notes:
    # - AI-generated code can help illustrate fault detection concepts, but it is limited.
    # - Expert human validation is required to verify logic, thresholds, and sensor behavior.
    # - Safety-critical aircraft systems must be reviewed, tested, and certified by authorities
    #   before they can be deployed in real operational environments.
    # - This simulation is not a replacement for certified maintenance software or engineering judgment.

    sensor_data = simulate_sensor_data()
    result = controller.analyze(sensor_data)
    if result:
        logging.info("Analysis completed with issue: %s", result)
    else:
        logging.info("Analysis completed successfully")


if __name__ == "__main__":
```

```
2026-04-08 10:34:01 [INFO] FaultDetectionSystem initialized with redundancy_threshold=2
2026-04-08 10:34:01 [WARNING] Detected faults: {'engine_temperature': 'Overheat', 'hydraulic_pressure': 'Low hydraulic pressure', 'battery_voltage': 'Low battery voltage', 'f
2026-04-08 10:34:01 [ERROR] CRITICAL ALERT: engine_temperature - Overheat
2026-04-08 10:34:01 [ERROR] CRITICAL ALERT: hydraulic_pressure - Low hydraulic pressure
2026-04-08 10:34:01 [ERROR] CRITICAL ALERT: battery_voltage - Low battery voltage
2026-04-08 10:34:01 [ERROR] CRITICAL ALERT: fuel_flow - Abnormal fuel flow
2026-04-08 10:34:01 [ERROR] Critical fault escalation
Traceback (most recent call last):
  File "c:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING\aircraft_maintenance_fault_detection.py", line 107, in analyze
    self.raise_alert(faults)
  File "c:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING\aircraft_maintenance_fault_detection.py", line 100, in raise_alert
    raise CriticalAlert("Critical aircraft maintenance fault detected")
CriticalAlert: Critical aircraft maintenance fault detected
2026-04-08 10:34:01 [INFO] Analysis completed with issue: Critical aircraft maintenance fault detected
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING>
```

Task 6 – Ethical Use of AI-Generated Code (Alternative Example)

Example Scenario:

Students analyze cases where AI-generated code is used in academic assignments or startups.

Generate a Python program that analyzes code submissions for potential ethical issues when using AI-generated code. Include checks for plagiarism, missing citations, and license compliance. Add features for logging, warnings, and validation. Include comments explaining the risks of using AI-generated code without understanding it, and demonstrate best practices such as proper citation, manual verification, and ethical reuse of AI outputs

```python
import logging
import re
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional

# Configure logging to preserve issue history and accountability.
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)


@dataclass
class AnalysisResult:
    plagiarism_score: float
    missing_citations: List[str]
    license_issues: List[str]
    warnings: List[str]
    valid: bool


class AICodeEthicsAnalyzer:
    """Analyzes code submissions for ethical issues in AI-generated outputs."""

    KNOWN_SNIPPETS = (
        "def quicksort(arr):",
        "SELECT * FROM",
        "class MyClass",
        "console.log('Hello World')",
    )

    LICENSE_HINTS = (
        "MIT": ("MIT License", "MIT"),
        "Apache-2.0": ("Apache License", "Apache-2.0"),
        "GPL": ("GNU General Public License", "GPL"),
    )

    CITATION_PATTERNS = (
        r"#\s*source:",
        r"#\s*citation:",
        r"#\s*reference:",
        r"//\s*source:",
        r"//\s*citation:",
    )

    def __init__(self, required_license: Optional[str] = None):
        self.required_license = required_license
        logging.info("AICodeEthicsAnalyzer initialized with required_license=%s", required_license)

    def load_code(self, path: Path) -> str:
        if not path.exists():
            raise FileNotFoundError(f"Code submission not found: {path}")
        if not path.is_file():
            raise ValueError(f"Path is not a file: {path}")

        text = path.read_text(encoding="utf-8")
        logging.debug("Loaded code from %s (%d characters)", path, len(text))
        return text

    def validate_code_text(self, text: str) -> None:
        if not text.strip():
            raise ValueError("Submission is empty")
        if len(text) < 20:
            raise ValueError("Submission is too short to evaluate meaningfully")
        logging.debug("Code text validated successfully")

    def check_plagiarism(self, text: str) -> float:
        # Simple heuristic: count known snippets and normalize by number of lines.
        matches = sum(1 for snippet in self.KNOWN_SNIPPETS if snippet in text)
        lines = max(1, len(text.splitlines()))
        score = matches / lines
        logging.info("Plagiarism heuristic score: %.1f (%d matches, %d lines)", score, matches, lines)
        return score

    def check_citations(self, text: str) -> List[str]:
        missing = []
        lines = text.splitlines()
        for i, line in enumerate(lines, start=1):
            if "TODO" in line or "Generated by AI" in line:
```

```python
class AICodeEthicsAnalyzer:

    def load_code(self, path: Path) -> str:
        if not path.exists():
            raise FileNotFoundError(f"Code submission not found: {path}")
        if not path.is_file():
            raise ValueError(f"Path is not a file: {path}")

        text = path.read_text(encoding="utf-8")
        logging.debug("Loaded code from %s (%d characters)", path, len(text))
        return text

    def validate_code_text(self, text: str) -> None:
        if not text.strip():
            raise ValueError("Submission is empty")
        if len(text) < 20:
            raise ValueError("Submission is too short to evaluate meaningfully")
        logging.debug("Code text validated successfully")

    def check_plagiarism(self, text: str) -> float:
        # Simple heuristic: count known snippets and normalize by number of lines.
        matches = sum(1 for snippet in self.KNOWN_SNIPPETS if snippet in text)
        lines = max(1, len(text.splitlines()))
        score = matches / lines
        logging.info("Plagiarism heuristic score: %.1f (%d matches, %d lines)", score, matches, lines)
        return score

    def check_citations(self, text: str) -> List[str]:
        missing = []
        lines = text.splitlines()
        for i, line in enumerate(lines, start=1):
            if "TODO" in line or "Generated by AI" in line:
                if not any(re.search(pattern, line, re.IGNORECASE) for pattern in self.CITATION_PATTERNS):
                    missing.append(f"line {i}: no citation for generated or placeholder content")
        logging.info("Found %d missing citation issues", len(missing))
        return missing

    def check_license_compliance(self, text: str) -> List[str]:
        issues = []
        header = "\n".join(text.splitlines()[:10])
        if self.required_license:
            if self.required_license not in header:
                issues.append(f"Required license '{self.required_license}' not declared in header")

        if not any(hint in header for hints in self.LICENSE_HINTS.values() for hint in hints):
            issues.append("No recognizable license header found")

        logging.info("License compliance issues: %s", issues)
        return issues

    def analyze_submission(self, path: Path) -> AnalysisResult:
        text = self.load_code(path)
        self.validate_code_text(text)

        plagiarism_score = self.check_plagiarism(text)
        missing_citations = self.check_citations(text)
        license_issues = self.check_license_compliance(text)

        warnings = []
        if plagiarism_score > 0.05:
            warnings.append("High similarity to known code patterns; investigate potential plagiarism")
        if missing_citations:
            warnings.append("Missing citations or source references for AI-generated content")
        if license_issues:
            warnings.append("License information may be incomplete or non-compliant")
        if "NOTICE" in text and "SPDX-License-Identifier" not in text:
            warnings.append("Explicit license notice present without SPDX identifier")

        valid = not (plagiarism_score > 0.05 or missing_citations or license_issues)
        if valid:
            logging.info("Submission passed ethical validation")
        else:
            logging.warning("Submission failed ethical validation")

        return AnalysisResult(
            plagiarism_score=plagiarism_score,
            missing_citations=missing_citations,
            license_issues=license_issues,
            warnings=warnings,
            valid=valid,
        )
```

```python
        missing_citations = self.check_citations(text)
        license_issues = self.check_license_compliance(text)

        warnings = []
        if plagiarism_score > 0.05:
            warnings.append("High similarity to known code patterns; investigate potential plagiarism")
        if missing_citations:
            warnings.append("Missing citations or source references for AI-generated content")
        if license_issues:
            warnings.append("License information may be incomplete or non-compliant")
        if "NOTICE" in text and "SPDX-License-Identifier" not in text:
            warnings.append("Explicit license notice present without SPDX identifier")

        valid = not (plagiarism_score > 0.05 or missing_citations or license_issues)
        if valid:
            logging.info("Submission passed ethical validation")
        else:
            logging.warning("Submission failed ethical validation")

        return AnalysisResult(
            plagiarism_score=plagiarism_score,
            missing_citations=missing_citations,
            license_issues=license_issues,
            warnings=warnings,
            valid=valid,
        )

    def report(self, result: AnalysisResult) -> None:
        logging.info("Analysis result: valid=%s", result.valid)
        for warning in result.warnings:
            logging.warning("Warning: %s", warning)
        for citation in result.missing_citations:
            logging.warning("Citation issue: %s", citation)
        for license_issue in result.license_issues:
            logging.warning("License issue: %s", license_issue)


def main() -> None:
    analyzer = AICodeEthicsAnalyzer(required_license="MIT")

    # Notes for reviewers:
    # - AI-generated code may contain plausible-looking logic but still be ethically unsound.
    # - Reviewers should not rely only on automated checks; manual verification is essential.
    # - Proper citation, license attribution, and understanding the output are best practices.
    # - Ethical reuse means acknowledging sources, respecting licenses, and avoiding blind copy-paste.
    # - This tool is illustrative, not a substitute for legal or compliance review.

    sample_path = Path("sample_submission.py")
    try:
        result = analyzer.analyze_submission(sample_path)
        analyzer.report(result)
        if not result.valid:
            logging.info("Address warnings before using or publishing this code")
        else:
            logging.info("Code submission is suitable for further review")
    except Exception as ex:
        logging.exception("Error analyzing submission: %s", ex)


if __name__ == "__main__":
    main()
```

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING> & "c:/Users/gbsgu/OneDrive/Documents/Desktop/Ai aSSISTED CODEING/.venv/Scri
2026-04-08 10:36:35 [INFO] AICodeEthicsAnalyzer initialized with required_license=MIT
2026-04-08 10:36:35 [ERROR] Error analyzing submission: Code submission not found: sample_submission.py
Traceback (most recent call last):
  File "c:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING\ai_code_ethics_analyzer.py", line 154, in main
    res                                                    ssion(sample_path)
                Open file in editor (ctrl + click)         ^^^^^^^^^^^^^^^^^^^
  File "c:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING\ai_code_ethics_analyzer.py", line 101, in analyze_submission
    text = self.load_code(path)
           ^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING\ai_code_ethics_analyzer.py", line 54, in load_code
    raise FileNotFoundError(f"Code submission not found: {path}")
FileNotFoundError: Code submission not found: sample_submission.py
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Ai aSSISTED CODEING>
```